

clusterAI

ciencia de datos en ingeniería industrial

UTN BA

curso I5521

clase_05: ML Workflow

Pipelines · ColumnTransformer · GridSearchCV

Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow — A. Géron

- Capítulo 2 → *End-to-End Machine Learning Project* (transformation pipelines, custom transformers)
- Capítulo 2 → *Fine-Tune Your Model* (Grid Search, Randomized Search)

Introduction to Statistical Learning (Python) — James, Witten, Hastie, Tibshirani

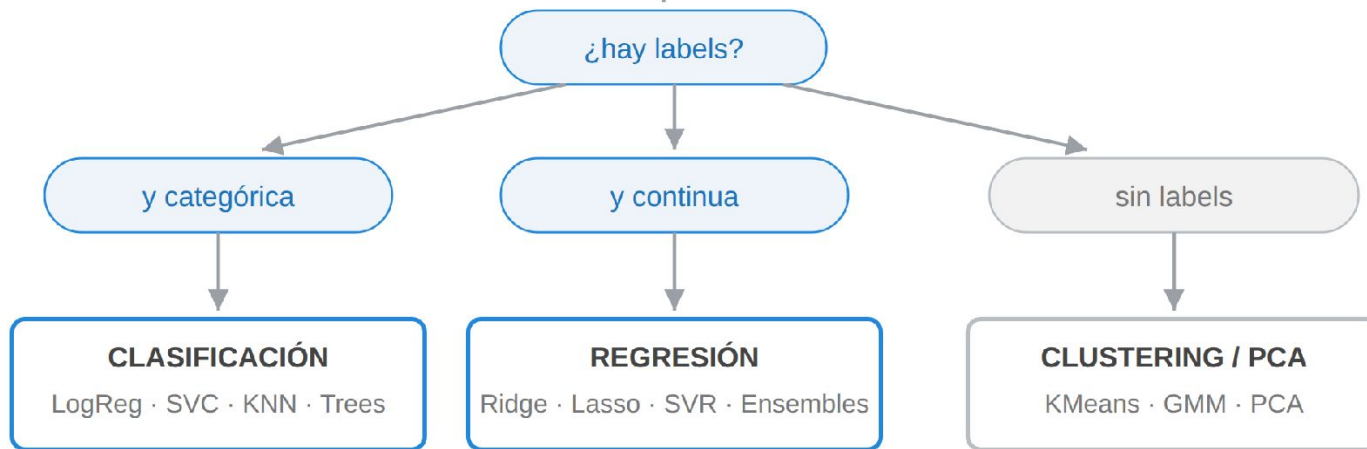
- Capítulo 5 → *Resampling Methods* (Cross-Validation)
- Capítulo 6 → *Model Selection and Regularization*

agenda clase05: ML Workflow

- Repaso: selección de modelos
- El problema del código "suelto" y el **data leakage**
- **Pipeline** → pasos secuenciales
- **ColumnTransformer** → pasos paralelos por columna
- Transformers propios: **FunctionTransformer** y clases custom
- Pipeline completo: preprocesamiento + modelo
- **Cross-validation** y **GridSearchCV** sobre el pipeline
- AutoML



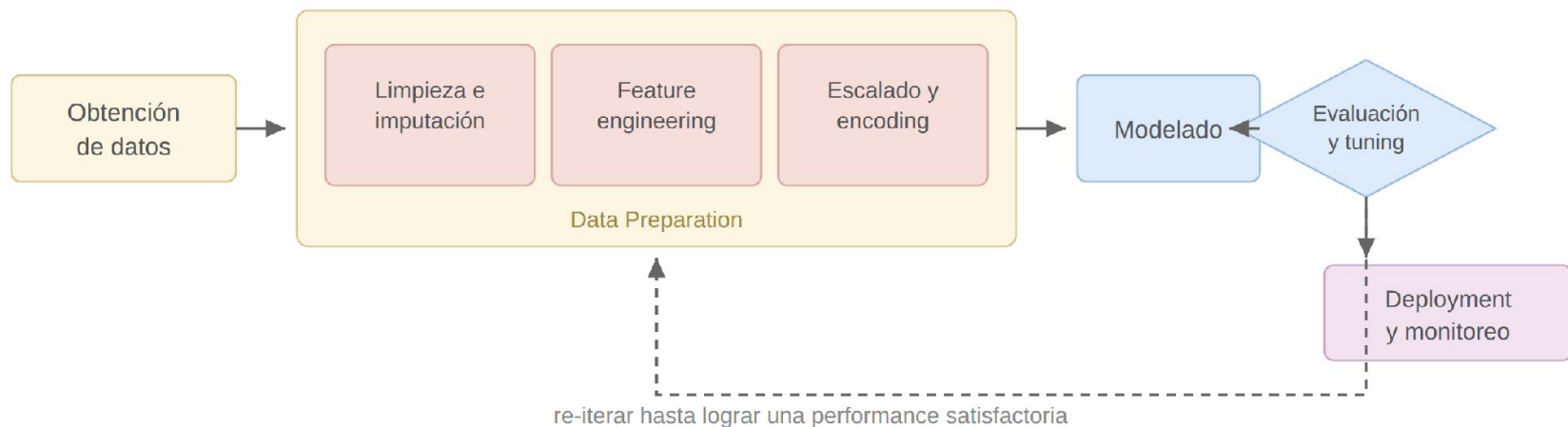
Repaso: selección de modelos





El problema: del notebook desordenado al pipeline

El workflow de un proyecto de ML



En la práctica el ciclo se recorre **decenas de veces**. Cada vuelta cambia una decisión de preprocesamiento o un hiper-parámetro: necesitamos que ese cambio se propague solo, sin re-ejecutar celdas a mano.

```
# celdas sueltas, ejecutadas en cualquier orden
imp = SimpleImputer(strategy='median')
X_tr['age'] = imp.fit_transform(X_tr[['age']])

scaler = StandardScaler()
X_tr[num] = scaler.fit_transform(X_tr[num])

ohe = OneHotEncoder()
cats = ohe.fit_transform(X_tr[cat])
X_tr = np.hstack([X_tr[num], cats.toarray()])

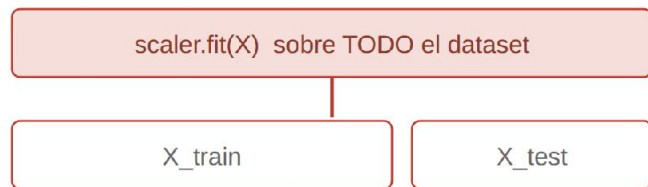
model = Ridge(alpha=1).fit(X_tr, y_train)
```

¿Qué sale mal?

- Hay que **repetir exactamente los mismos pasos, en el mismo orden**, sobre `X_test` y sobre cada dato nuevo en producción.
- Los objetos ajustados (`imputer`, `scaler`, `ohe`) quedan sueltos en el notebook: hay que guardarlos uno por uno.
- No se puede cruzar-validar: el `cross_val_score` recibiría datos **ya transformados**.
- Re-ejecutar celdas fuera de orden corrompe `X_train` en silencio.
- Y el peor de todos → **data leakage**.

Data leakage: el error silencioso

MAL



la media y el desvío ya "vieron" el test
→ el score de validación es optimista

BIEN



los parámetros se aprenden SOLO en train

Data leakage = información del conjunto de evaluación se filtra al entrenamiento. El modelo parece excelente en validación y falla en producción. Ocurre con escalado, imputación, encoding, selección de features y *target encoding*.

En cross-validation el problema se multiplica: **en cada fold** el preprocesamiento debe re-ajustarse usando sólo las K-1 particiones de entrenamiento.



Pipeline: encadenar pasos en secuencia

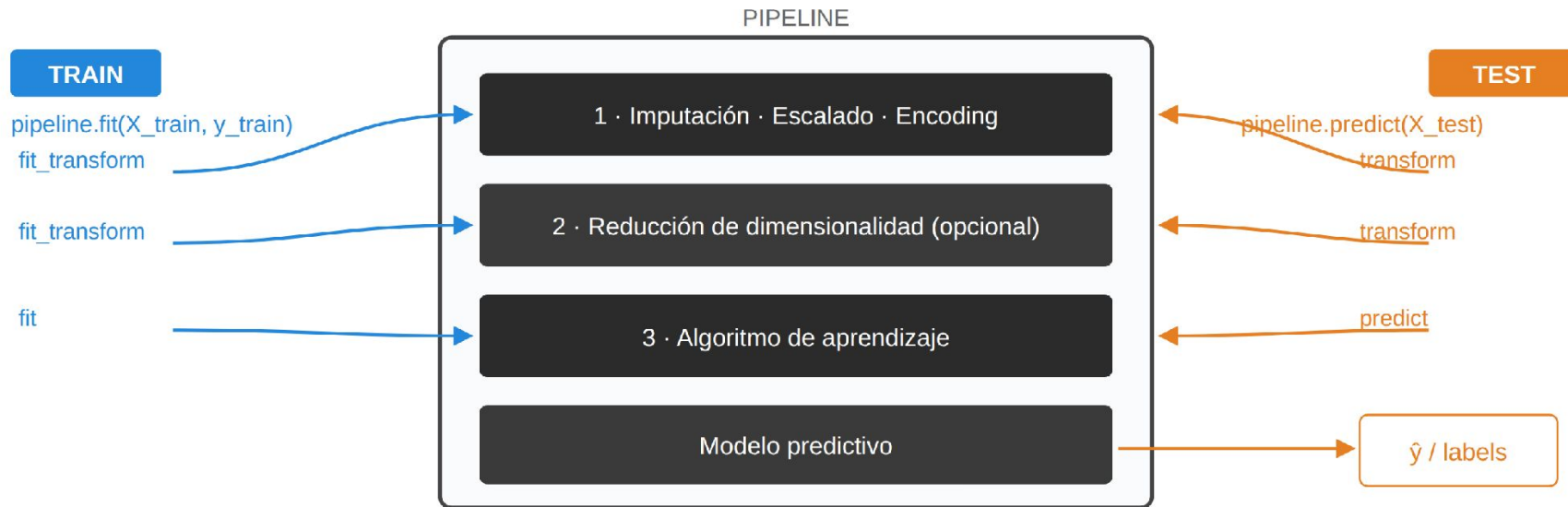
¿Qué es un Pipeline?

Un **Pipeline** es una cadena de operaciones de un proyecto de ML (preprocesamiento, entrenamiento, predicción) encapsulada en **un único objeto**.

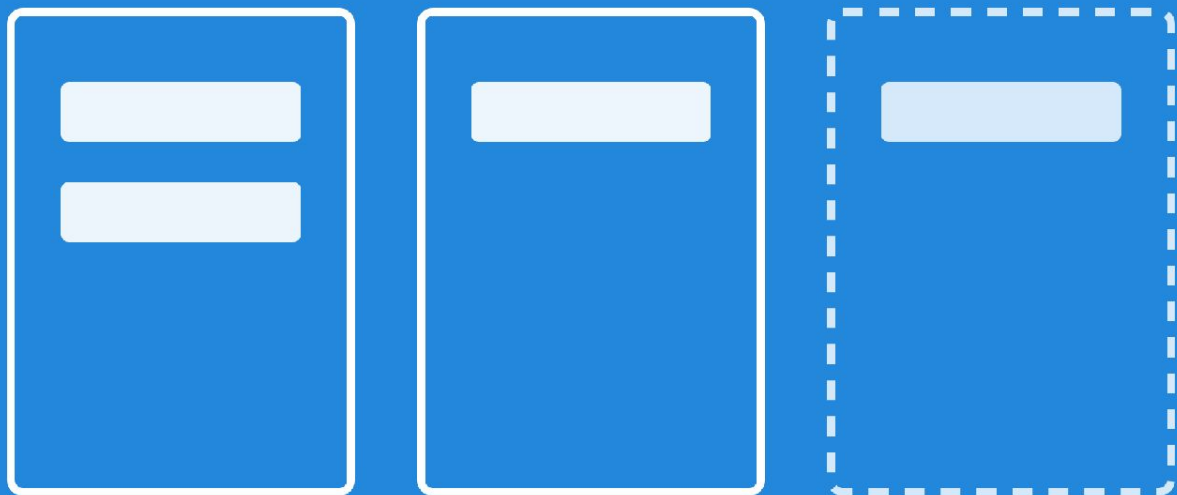
- **Legibilidad:** todo el flujo se lee en una sola celda.
- **Orden garantizado:** los pasos se ejecutan siempre igual, en train y en test.
- **Sin leakage:** `fit` sólo toca los datos de entrenamiento.
- **Reproducible y deployable:** se serializa un único artefacto.
- **Optimizable:** el pipeline entero entra a `GridSearchCV`.

Método	Transformer	Estimador (modelo)
<code>.fit(X, y)</code>	Aprende parámetros de la transformación (media, desvío, categorías, mediana)	Aprende los parámetros del modelo (w , β)
<code>.transform(X)</code>	Aplica lo aprendido a datos nuevos	—
<code>.fit_transform(X)</code>	Ambos, en un paso (sólo para train)	—
<code>.predict(X)</code>	—	Devuelve $\hat{y}$
<code>.score(X, y)</code>	—	Métrica por defecto (R^2 , accuracy)

Pipeline: train vs test



Con `.fit()` los transformers **aprenden y guardan** sus parámetros dentro del pipeline. Con `.predict()` sólo se ejecuta `.transform()` reusando lo aprendido.



ColumnTransformer: transformaciones en paralelo

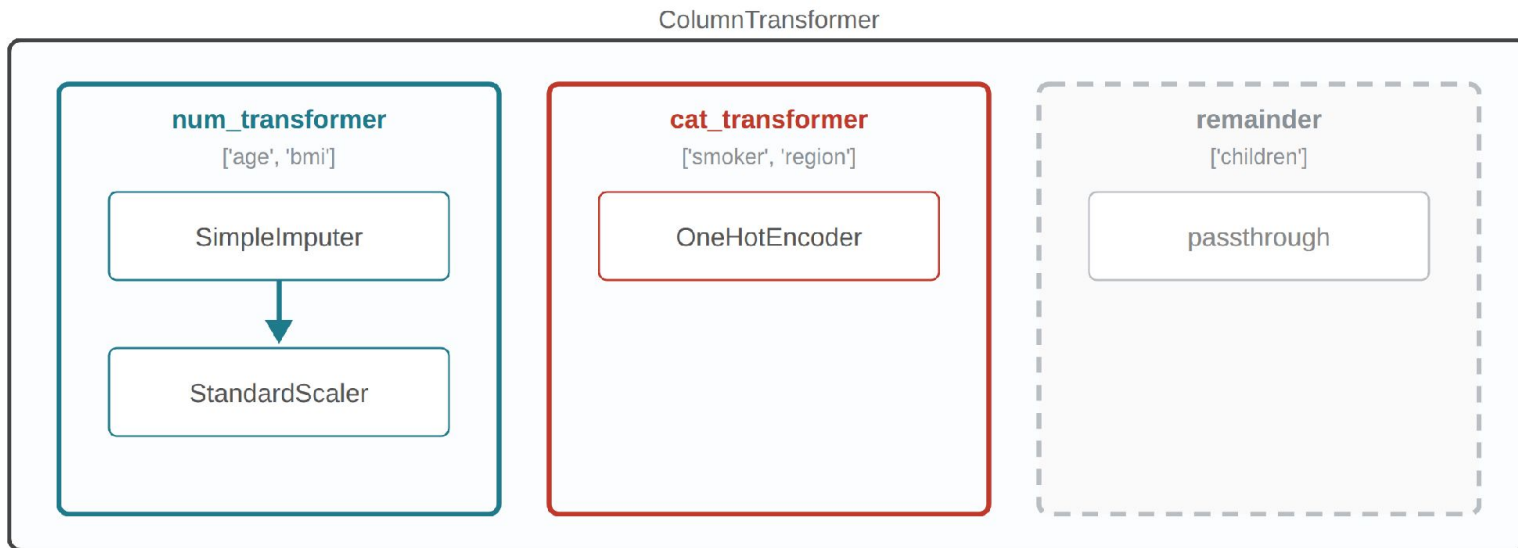
El problema: columnas heterogéneas

Un Pipeline aplica **los mismos pasos a todas las columnas**. Pero un dataset real es heterogéneo: no se puede escalar una columna de texto ni one-hot-encodear la edad.



El **ColumnTransformer** aplica transformaciones **distintas** a **subconjuntos distintos** de columnas, **en paralelo**, y luego concatena los resultados horizontalmente.

ColumnTransformer: el esquema



`salida = np.hstack([ numéricas , categóricas , resto ])`

Un **Pipeline** puede ir **adentro** de un `ColumnTransformer`: secuencia dentro de paralelo.

b) ColumnTransformer: código

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder

# Secuencia para las numéricas: imputar -> escalar
num_transformer = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler', StandardScaler())])

cat_transformer = OneHotEncoder(handle_unknown='ignore')

# Se paralelizan: cada transformer con SU lista de columnas
preprocessor = ColumnTransformer([
    ('num_transformer', num_transformer, ['age', 'bmi']),
    ('cat_transformer', cat_transformer, ['smoker', 'region'])])
```

Cada elemento es una tupla de **tres**: ('nombre', transformer, columnas) .

`handle_unknown='ignore'` evita que el pipeline se detenga si aparece una categoría nueva en test o en producción.

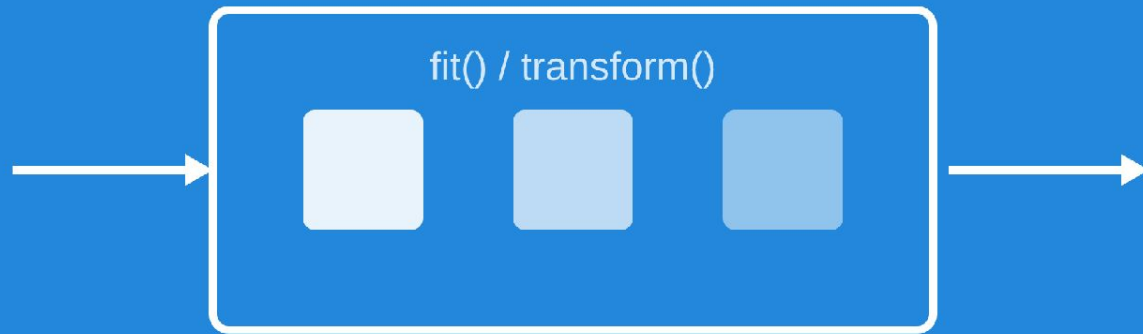
remainder: qué pasa con el resto

Las columnas no mencionadas se **descartan** por defecto (`remainder='drop'`). Para conservarlas sin tocar:

```
preprocessor = ColumnTransformer([\n    ('num_transformer', num_transformer, ['age', 'bmi']),\n    ('cat_transformer', cat_transformer, ['smoker', 'region'])],\n    remainder='passthrough')      # 'children' sobrevive intacta\n\n# También puede pasarse un transformer para el resto:\n# remainder=SimpleImputer(strategy='most_frequent')
```

remainder	Efecto sobre las columnas no listadas
'drop' (default)	Se eliminan de la salida
'passthrough'	Se concatenan sin modificar, al final
un transformer	Se les aplica esa transformación

Cuidado con el orden: las columnas de `passthrough` quedan siempre **al final**, no en su posición original.



FunctionTransformer

Permite **encapsular una función de Python** dentro de un transformer de scikit-learn, para poder usarla en un Pipeline o en un ColumnTransformer.

```
from sklearn.preprocessing import FunctionTransformer

rounder = FunctionTransformer(lambda arr: np.round(arr, decimals=2))

num_transformer = Pipeline([
    ('imputer', SimpleImputer()),
    ('scaler', StandardScaler()),
    ('rounder', rounder)])      # se enchufa como un paso más
```

Usos típicos

log / sqrt, clipping, redondeo, combinar columnas,
pasar de array a DataFrame.

Limitación

Sólo sirve para transformaciones **stateless**: no
puede aprender nada en `.fit()`.

Stateless vs stateful

Stateless — no guarda nada durante `.fit()`. La transformación de una muestra no depende del resto del dataset.

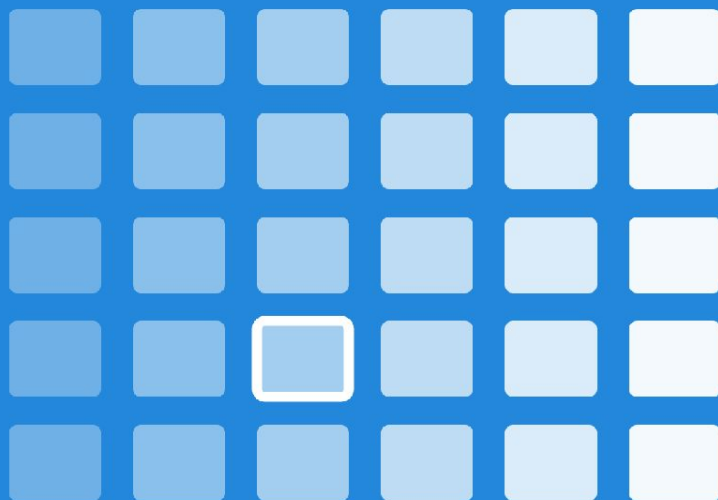
- $X \rightarrow \log(X)$
- $(X_1, X_2) \rightarrow X_1 + 5 \cdot X_2$
- redondeos, clipping, cambios de unidad

FunctionTransformer alcanza.

Stateful — aprende algo en `.fit(X_train)` y lo **reutiliza** en `.transform(X_test)`.

- StandardScaler $\rightarrow$ aprende μ y σ del train
- MinMaxScaler $\rightarrow$ aprende $X_{\min}$, $X_{\max}$
- SimpleImputer $\rightarrow$ aprende la media/mediana
- OneHotEncoder $\rightarrow$ aprende las categorías

Hay que escribir una clase propia.



Parámetros vs hiper-parámetros

Parámetros (w , β)

Los **aprende el algoritmo** durante `.fit()` minimizando la función de costo.

Ej.: los coeficientes de una regresión

Hiper-parámetros

Los **fija el usuario antes** de entrenar. Definen la familia de funciones y la **complejidad** del modelo.

Ej.: `alpha`, `C`, `max_depth`, `k` de KNN.

Y ahora, gracias al pipeline, también son hiper-parámetros:

- la **estrategia de imputación** (`'mean'` vs `'median'`)
- el **tipo de escalado** (Standard vs MinMax vs Robust vs ninguno)
- si se aplica o no **PCA**, y con cuántas componentes
- qué features se construyen

Al encapsular todo en un Pipeline, las **decisiones de preprocesamiento dejan de ser artesanales** y pasan a optimizarse con evidencia, igual que los del modelo.

Cross-validation: repaso

Se divide el **training set** en K folds y se itera K veces: en cada iteración un fold funciona como validación y los K-1 restantes como entrenamiento.

Se promedian las K métricas → estimación

estadísticamente más robusta del error en datos no vistos, con su desvío.

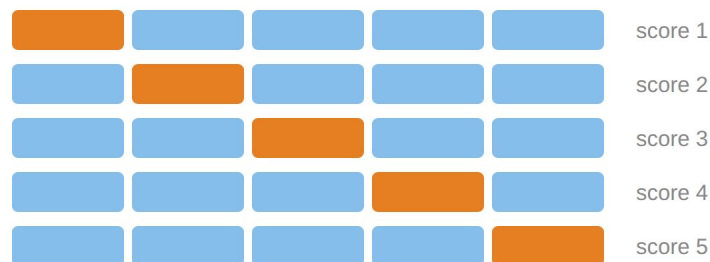
Permite comparar configuraciones de hiper-parámetros **sin tocar el test set**.

El **test set** se reserva hasta el final. Si se lo usa para elegir hiper-parámetros, deja de ser una estimación honesta.

Dataset completo



K-Fold sobre el train



score final = media(score 1..5) ± desvío

Grid Search: la grilla

Se define una **grilla** de valores posibles para cada hiper-parámetro. Para **cada combinación** se corre una cross-validation completa y se guarda la media del score. Gana la combinación con mejor media.

	alpha				
	0.1	0.5	1	5	10
'mean'	.712	.731	.747	.739	.722
'median'	.708	.728	.743	.736	.719

imputer__strategy

mejor combinación: strategy='mean', alpha=1 → R^2 CV = 0.747

5 alphas × 2 estrategias × 5 folds = 50 entrenamientos

GridSearchCV sobre el Pipeline

```
from sklearn.model_selection import GridSearchCV

grid_search = GridSearchCV(
    pipeline,                                     # el pipeline COMPLETO
    param_grid={
        # un hiper-parámetro del PREPROCESAMIENTO
        'columntransformer__pipeline__simpleimputer__strategy': ['mean', 'median'],
        # un hiper-parámetro del MODELO
        'ridge__alpha': [0.1, 0.5, 1, 5, 10]},
    cv=5,
    scoring='r2',
    n_jobs=-1,                                   # paraleliza en todos los cores
    refit=True)                                  # re-entrena el mejor sobre TODO el train (default)
```

Esto es lo que hace valioso al pipeline: se optimizan **a la vez** la estrategia de imputación y la regularización del modelo, sin leakage y en **una sola celda**.

RandomizedSearchCV

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import loguniform

search = RandomizedSearchCV(
    pipeline,
    param_distributions={
        'ridge__alpha': loguniform(1e-3, 1e3)}, # distribución continua
    n_iter=60, # presupuesto fijo de combinaciones
    cv=5, scoring='r2', n_jobs=-1, random_state=42)
```

También se puede buscar **entre modelos distintos** pasando una **lista de grillas**:

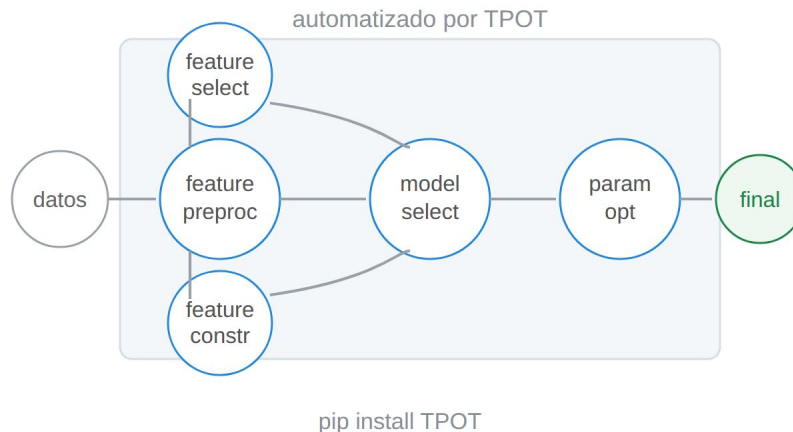
```
param_grid = [
    {'model': [Ridge()], 'model__alpha': [0.1, 1, 10]},
    {'model': [Lasso()], 'model__alpha': [0.01, 0.1, 1]},
    {'model': [RandomForestRegressor()], 'model__max_depth': [5, 10, None]}
```

Truco: nombrar el último paso '`model`' para poder reemplazarlo desde la grilla.

Bonus: AutoML (TPOT)

TPOT (Tree-based Pipeline Optimization Tool) busca automáticamente, con programación genética, la combinación de preprocesamiento + modelo + hiper-parámetros que maximiza el score de CV.

- Exporta el pipeline ganador como **código Python legible**.
 - Útil como **baseline fuerte** y para explorar familias de modelos.
 - Otras opciones: auto-sklearn, FLAML, H2O AutoML.
- No reemplaza el criterio: no entiende el negocio, no detecta leakage conceptual ni features mal construidas, y es muy costoso en cómputo.



Checklist del proyecto de ML supervisado

Datos	Cargar y validar el dataset. Analizar distribuciones, outliers, NaN. Nunca mirar el test hasta el final.
Preprocesamiento	Definir el ColumnTransformer: imputar → escalar → encodear. Todo dentro del Pipeline.
Baseline	Entrenar un modelo simple (media, regresión lineal) para tener un score de referencia.
Exploración de modelos	Cross-validar con cv=5 al menos 2-3 familias de modelos. Comparar el score de CV, no el de train.
Feature Engineering	Construir features derivadas. Agregar con FunctionTransformer o clase custom. Re-crossvalidar.
Hyperparameter Tuning	GridSearchCV / RandomizedSearchCV sobre el pipeline completo. Registrar best_params_ y best_score_.
Evaluación final	Correr el best_estimator_ sobre el test set UNA sola vez. Reportar RMSE, MAE y R^2 .